

Data Mining Concepts and Techniques (CS24303) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank (46 Topics)

Module I: Introduction to Data Mining & Data Attributes — 14 Topics Syllabus

- | | |
|---|--|
| 1. Introduction to Data Mining & KDD | 2. Relational Databases Architecture |
| 3. Data Warehouses Fundamentals | 4. Transactional Databases |
| 5. Advanced Database Systems & Applications | 6. Data Mining Functionalities (6 Types) |
| 7. Classification of Data Mining Systems | 8. Major Issues & Challenges in Data Mining |
| 9. Data & Dataset Anatomy | 10. Data Objects & 4 Attribute Types |
| 11. Basic Statistical Descriptions of Data | 12. Data Visualization Techniques (5 Paradigms) |
| 13. Measuring Data Similarity (Cosine, Jaccard) | 14. Measuring Data Dissimilarity (L1, L2, Lp, Mixed) |

Topic 1: Introduction to Data Mining & The Knowledge Discovery (KDD) Process

Data Mining is the computational process of discovering hidden, previously unknown, valid, novel, potentially useful, and understandable patterns, correlations, anomalies, and actionable knowledge from massive volumes of structured, semi-structured, and unstructured data repositories.

Often called *Knowledge Discovery in Databases (KDD)*, data mining represents the essential algorithmic step within a broader, highly iterative multi-phase engineering pipeline designed to convert raw operational data into high-value strategic decision models.



FIGURE 1.1: THE END-TO-END KNOWLEDGE DISCOVERY IN DATABASES (KDD) PIPELINE

Comprehensive Analysis of the 7 KDD Phases:

- 1. Data Cleaning:** Industrial sensors, transactional points-of-sale, and web forms produce data corrupted by transmission loss, hardware malfunctions, or transcription mistakes. Cleaning eliminates extreme noise and handles missing attributes via statistical imputation.
- 2. Data Integration:** Merges disparate heterogeneous data sources into a unified repository. Resolves semantic conflicts, entity identification problems (e.g., matching `Cust\_ID` with `Client\_Number`), and detects attribute redundancy through correlation analysis.
- 3. Data Selection:** Dynamically isolates task-relevant attributes and subsets required for a specific business query from the massive data lake or enterprise warehouse.
- 4. Data Transformation:** Normalizes numeric scales ($[0, 1]$ Min-Max or Z-Score), aggregates granular records into summary facts, and constructs concept hierarchies for multi-level pattern mining.
- 5. Data Mining (Core Engine):** Employs machine learning, statistical pattern recognition, and database search algorithms (such as Apriori, FP-Growth, Decision Trees, K-Means, Neural Networks) to discover recurring structures, decision boundaries, or clusters.
- 6. Pattern Evaluation:** Evaluates discovered candidates against rigorous mathematical interestingness metrics (Support, Confidence, Lift, Information Gain, Silhouette Coefficients) to filter out trivial correlations and avoid statistical overfitting.
- 7. Knowledge Presentation:** Uses visual analytics, scatter matrices, parallel coordinate graphs, and interactive decision trees to present discoveries in an interpretable format for human domain experts.

Topics 2 – 5: Data Repositories & Advanced Database Architectures

Data mining algorithms must adapt their indexing, traversal, and caching strategies according to the specific data storage architecture:

Repository Type	Architecture & Data Organization	Data Mining Use Cases & Algorithmic Adaptations
2. Relational Databases (RDBMS)	Organized as structured collections of normalized tables linked by foreign keys, accessed via SQL queries. Compliant with ACID properties (Atomicity, Consistency, Isolation, Durability).	Relational association mining, inductive logic programming (ILP), classification of multi-table customer records without creating expensive flat denormalized views.
3. Data Warehouses	Subject-oriented, integrated, time-variant, non-volatile repositories modeled using multidimensional Data Cubes , Fact tables, and Dimension tables for Online Analytical Processing (OLAP).	On-Line Analytical Mining (OLAM), roll-up/drill-down exploration, discovery-driven data cube navigation using statistical anomaly indicators.
4. Transactional Databases	Flat files or nested tables where each record represents a distinct transaction consisting of a unique identifier ('TID') and a set of items purchased simultaneously ($T \subseteq I$).	Frequent itemset mining, cross-selling analysis, market basket analysis, sequential traversal algorithms (Apriori, FP-Growth, ECLAT).
5. Spatial Databases	Stores multidimensional geometric objects (points, lines, polygons, raster grids) indexed via R-Trees, Quad-trees, or k-d trees with GIS coordinate systems.	Spatial autocorrelation analysis, spatial trend mining, density-based geographic clustering (DBSCAN), trajectory tracking.
5. Time-Series & Temporal	Stores time-stamped numeric sequence values recorded at uniform or non-uniform intervals (stock quotes, IoT sensor telemetry, ECG).	Trend analysis, cyclic/seasonal decomposition, Dynamic Time Warping (DTW), autoregressive ARIMA forecasting, motif discovery.
5. Text & Web Databases	Unstructured text documents, HTML web graphs with hyperlinks, XML trees, server clickstream logs, social media graphs.	TF-IDF vector space modeling, Latent Dirichlet Allocation (LDA), PageRank link analysis, sentiment analysis, web usage mining.

Topic 6: Data Mining Functionalities (What Kinds of Patterns Can Be Mined?)

Data mining tasks are broadly divided into two major complementary paradigms: **Descriptive** (summarizing and discovering structural properties of current data) and **Predictive** (building statistical inference models to forecast future outcomes).

Mining Functionality	Paradigm	Mathematical / Algorithmic Basis	Industrial Example
1. Characterization & Discrimination	Descriptive	Summarizing general features of a target class (Characterization) vs. contrasting it against comparative classes (Discrimination) via Attribute-Oriented Induction.	Summarizing characteristics of top 10% spending customers vs. comparing them against occasional buyers.
2. Frequent Patterns, Associations & Correlations	Descriptive	Discovering itemsets appearing with support $\geq \text{min_sup}$ and confidence $\geq \text{min_conf}$. Generating rules $A \Rightarrow B$ satisfying $\text{Lift}(A, B) > 1$.	$\text{buys}(\text{laptop}) \Rightarrow \text{buys}(\text{mouse})$ [2%, 75%].
3. Classification	Predictive	Supervised learning model constructed from labeled training tuples to predict discrete categorical class labels (Decision Trees, Naïve Bayes, SVM, Neural Networks).	Classifying loan applicants as <i>Safe</i> or <i>Risky</i> based on credit score, salary, and debt.
4. Regression	Predictive	Supervised learning mapping data objects to continuous real-valued numerical variables (Linear Regression, Non-linear curve fitting, Ridge/Lasso).	Predicting next quarter's revenue based on advertising expenditure and historical growth.
5. Cluster Analysis	Descriptive	Unsupervised learning grouping objects into clusters such that intra-cluster similarity is maximized and inter-cluster similarity is minimized (K-Means, Hierarchical, DBSCAN).	Discovering distinct customer market segments without predefined class labels.
6. Outlier Analysis (Anomaly Detection)	Descriptive / Predictive	Identifying data objects that deviate significantly from general distribution patterns (Statistical discordance tests, Distance-based, Density-based LOF).	Credit card fraud detection; sudden spike in foreign transactions at 3 AM.

Topic 7: Classification of Data Mining Systems

Data mining systems are categorized across 4 multidimensional axes:

1. **By Data Models Mined:** Relational, Transactional, Object-Oriented, Spatial, Temporal, Time-Series, Text, Web, Graph, Multimedia, and Streaming Data Mining systems.
2. **By Types of Knowledge Discovered:** Association Rule systems, Classification & Prediction engines, Clustering systems, Sequential Pattern miners, Outlier detectors, and Deep Representation embedding systems.
3. **By Underlying Mining Techniques:** Database-oriented (SQL/OLAP-driven), Machine learning, Statistics, Information retrieval, Neural networks, Genetic algorithms, and Visual analytics.
4. **By Applications Adapted:** Telecommunications, Healthcare, Banking & Finance, Bioinformatics, E-Commerce, Retail, and Intrusion Detection.

Topic 8: Major Issues & Challenges in Data Mining

5 Major Technical Challenges in Modern Data Mining

1. **Mining Methodology & User Interaction:** Mining complex knowledge in multidimensional space; integrating background domain knowledge; designing expressive mining query languages; intuitive visual representation.
2. **Performance, Scalability & Parallelism:** Processing massive petabyte data streams within strict runtime constraints ($O(n)$ or sub-linear complexity); parallel, distributed, and incremental mining algorithms.
3. **Handling Data Diversity & High Dimensionality:** Dealing with noisy, incomplete data; handling extreme sparsity; mitigating the curse of dimensionality where distance metrics lose discriminative power.
4. **Pattern Evaluation & Overfitting:** Separating genuine causal signals from random noise; addressing the multiple hypothesis testing problem in large candidate spaces.
5. **Social Impacts & Privacy Preservation:** Ensuring algorithmic fairness, differential privacy, GDPR compliance, and avoiding bias in automated credit/hiring algorithms.

Topics 9 – 10: Data Objects & 4 Fundamental Attribute Types

A dataset is a collection of **Data Objects** (also known as samples, records, points, vectors, tuples, observations, or instances). A data object represents an individual entity. Data objects are described by a set of **Attributes** (also called features, dimensions, variables, or fields).

Attribute Class	Mathematical Properties	Permissible Operations	Concrete Examples
1. Nominal (Categorical)	Values are distinct symbols or names without any inherent ordering or magnitude.	Equality ($=$), Inequality ($\neq$), Mode, Frequency counting. (No mean or median).	'Marital_Status' $\in$ {Single, Married, Divorced}, 'Eye_Color', 'Zip_Code'.
2. Binary (Special Nominal)	Nominal attribute with exactly 2 states (0 and 1, or True and False). Symmetric: Both outcomes equally important (e.g., 'Gender' M/F). Asymmetric: Presence (1) is far more significant/rare than absence (0) (e.g., 'Medical_Test' +/−).	Simple Matching Coefficient (SMC) for symmetric; Jaccard Coefficient for asymmetric.	'Covid_Test' $\in$ {0, 1}, 'Spam_Flag' $\in$ {0, 1}, 'Item_Purchased'.
3. Ordinal	Values have a meaningful sequential order or ranking, but the distance/difference between successive values cannot be measured.	Order comparison ($<$, $\leq$, $>$, $\geq$), Median, Percentiles, Rank correlation (Spearman's ρ).	'Customer_Satisfaction' $\in$ {Poor, Fair, Good, Excellent}, 'Academic_Grade' (A, B, C, D, F).
4. Numeric: Interval-Scaled	Measured on a scale of equal-sized units. Meaningful differences between values, but no true zero point (arbitrary zero).	Addition (+), Subtraction (−), Mean, Standard Deviation. (Ratios are meaningless).	'Temperature' in °C or °F (0°C does not mean zero heat; 20°C is not twice as hot as 10°C), 'Calendar_Year'.
4. Numeric: Ratio-Scaled	Numeric attribute with an absolute, inherent true zero point (0.0 = none).	All arithmetic operations (+, −, $\times$, $\div$), Geometric Mean, Harmonic Mean, Coefficient of Variation.	'Salary' (\0 means zero earnings; \$100k is exactly 2\times\$50k), 'Age', 'Weight', 'Length', 'Price'.

Topic 11: Basic Statistical Descriptions of Data

To understand the fundamental characteristics, central tendency, and dispersion of numeric data, we compute descriptive summary statistics.

1. Measures of Central Tendency

- **Arithmetic Mean:** $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$. Sensitive to extreme outliers.
- **Trimmed Mean:** Computed after discarding a fixed percentage (e.g., 5%) of the lowest and highest values to improve robustness against outliers.
- **Median:** Middle value in sorted order. For even n , average of middle two. Robust measure of central tendency:

$$\text{Median} = \begin{cases} x_{(n+1)/2} & \text{if } n \text{ is odd} \\ \frac{x_{n/2} + x_{(n/2)+1}}{2} & \text{if } n \text{ is even} \end{cases}$$

- **Mode:** The most frequently occurring value in the dataset. A dataset can be unimodal, bimodal, multimodal, or have no mode.
- **Midrange:** Average of smallest and largest values: $\text{Midrange} = \frac{\min + \max}{2}$.

Memory Key: Skewness & Mean vs. Median Relationship

Symmetric Distribution: Mean = Median = Mode.

Positively Skewed (Right-Skewed): Tail extends to the right; $\text{Mode} < \text{Median} < \text{Mean}$.

Negatively Skewed (Left-Skewed): Tail extends to the left; $\text{Mean} < \text{Median} < \text{Mode}$.

2. Measures of Dispersion & The Five-Number Summary

- **Range:** $\text{Range} = \max(x) - \min(x)$.
- **Quartiles:** Q_1 (25th percentile / First Quartile), Q_2 (50th percentile / Median), Q_3 (75th percentile / Third Quartile).
- **Interquartile Range (IQR):** Distance between 75th and 25th percentiles:

$$\text{IQR} = Q_3 - Q_1$$

- **Five-Number Summary:** $[\min, Q_1, \text{Median}, Q_3, \max]$.
- **Boxplot Outlier Boundary Rule (Tukey's Rule):** Any data point falling beyond the inner fences is flagged as an outlier:

$$\text{Lower Fence} = Q_1 - 1.5 \times \text{IQR}, \quad \text{Upper Fence} = Q_3 + 1.5 \times \text{IQR}$$

- **Sample Variance (s^2) & Standard Deviation (s):**

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}$$



Step-by-Step Solved Problem: Five-Number Summary & Outlier Detection

Dataset (Sorted): $S = [12, 15, 18, 22, 24, 25, 28, 30, 32, 35, 38, 42, 45, 95]$ ($n = 14$).

Calculations:

1. $\min = 12, \quad \max = 95$.
2. $\text{Median } (Q_2) = \frac{x_7 + x_8}{2} = \frac{28 + 30}{2} = \mathbf{29.0}$.
3. $Q_1 = \text{Median of lower half } [12, 15, 18, 22, 24, 25, 28] = \mathbf{22.0}$.
4. $Q_3 = \text{Median of upper half } [30, 32, 35, 38, 42, 45, 95] = \mathbf{38.0}$.
5. $\text{IQR} = Q_3 - Q_1 = 38.0 - 22.0 = \mathbf{16.0}$.
6. $\text{Lower Fence} = 22.0 - 1.5(16.0) = 22.0 - 24.0 = \mathbf{-2.0}$.
7. $\text{Upper Fence} = 38.0 + 1.5(16.0) = 38.0 + 24.0 = \mathbf{62.0}$.
8. **Outlier Identification:** Value $95 > 62.0 \implies \mathbf{95}$ is confirmed as an extreme outlier!

Topic 12: Data Visualization Techniques

Visualization Category	Core Visual Mapping Mechanism	Common Visual Types
1. Pixel-Oriented Techniques	Maps each attribute value to a colored pixel; arranges pixels in recursive space-filling curve windows.	Space-filling curve pixel windows, Circle segments.
2. Geometric Projection	Projects multi-dimensional datasets onto 2D/3D visual subspaces.	Scatter Plot Matrices, Parallel Coordinates (attributes mapped to parallel vertical axes; tuples drawn as polylines).
3. Icon-Based Techniques	Maps attribute dimensions to visual facial or shape features of graphic icons.	Chernoff Faces (eye size, smile curvature, head shape), Stick figures.
4. Hierarchical Techniques	Subdivides attribute space into hierarchical nested subspaces.	Dimensional Stacking, Treemaps (nested rectangular area proportional to value).
5. Visualizing Complex Data	Renders topological networks, text tags, and geographical thematic maps.	Node-link graphs, Chord diagrams, Word clouds, Choropleth maps.

Topics 13 – 14: Measuring Data Similarity and Dissimilarity

Proximity between two objects $x = (x_1, x_2, \dots, x_p)$ and $y = (y_1, y_2, \dots, y_p)$ is quantified using **Distance (Dissimilarity)** $d(x, y)$ or **Similarity** $s(x, y)$.

1. Proximity Measures for Numeric Attributes

Minkowski Distance (L_p Norm):
$$d(x, y) = \left(\sum_{i=1}^p |x_i - y_i|^p \right)^{1/p}$$

Metric Name	Parameter	Mathematical Formula	Properties & Geometric Interpretation
Manhattan Distance (L_1)	$p = 1$	$d(x, y) = \sum_{i=1}^p x_i - y_i $	City-block grid distance. Robust against single-feature extreme outliers.
Euclidean Distance (L_2)	$p = 2$	$d(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$	Standard straight-line Euclidean distance in Cartesian space.
Supremum Distance (L_∞)	$p \rightarrow \infty$	$d(x, y) = \max_{i=1}^p x_i - y_i $	Maximum coordinate difference along any single dimension (Chebyshev distance).

2. Proximity for Asymmetric Binary & Document Vector Data

Metric Name	Mathematical Formulation	Core Use Case
Simple Matching Coeff. (SMC)	$SMC(x, y) = \frac{q + t}{q + r + s + t}$	Symmetric binary data (where 0 – 0 matches t are as important as 1 – 1 matches q).
Jaccard Coefficient (J)	$J(x, y) = \frac{ x \cap y }{ x \cup y } = \frac{q}{q + r + s}, \quad d_J(x, y) = 1 - J(x, y) = \frac{r + s}{q + r + s}$	Asymmetric binary data (ignores joint absence t ; $q = 1 - 1, r = 1 - 0, s = 0 - 1$).
Cosine Similarity	$sim(x, y) = \frac{\vec{x} \cdot \vec{y}}{\ \vec{x}\ \ \vec{y}\ } = \frac{\sum_{i=1}^p x_i y_i}{\sqrt{\sum x_i^2} \sqrt{\sum y_i^2}}$	Text Mining & TF-IDF Vectors: Measures angle θ between vectors; invariant to document length.

Step-by-Step Solved Problem: Proximity Calculations on Vector Data

Given Two Data Points: $x = (3, 0, 4, 1)$, $y = (1, 2, 4, 0)$.

1. Manhattan Distance (L_1):

$$d(x, y) = |3 - 1| + |0 - 2| + |4 - 4| + |1 - 0| = 2 + 2 + 0 + 1 = \mathbf{5.0}$$

2. Euclidean Distance (L_2):

$$d(x, y) = \sqrt{(3 - 1)^2 + (0 - 2)^2 + (4 - 4)^2 + (1 - 0)^2} = \sqrt{4 + 4 + 0 + 1} = \sqrt{9} = \mathbf{3.0}$$

3. Supremum Distance (L_∞):

$$d(x, y) = \max(|3 - 1|, |0 - 2|, |4 - 4|, |1 - 0|) = \max(2, 2, 0, 1) = \mathbf{2.0}$$

4. Cosine Similarity ($\text{sim}(x, y)$):

$$\vec{x} \cdot \vec{y} = (3 \times 1) + (0 \times 2) + (4 \times 4) + (1 \times 0) = 3 + 0 + 16 + 0 = 19$$

$$\|\vec{x}\| = \sqrt{3^2 + 0^2 + 4^2 + 1^2} = \sqrt{9 + 0 + 16 + 1} = \sqrt{26} \approx 5.099$$

$$\|\vec{y}\| = \sqrt{1^2 + 2^2 + 4^2 + 0^2} = \sqrt{1 + 4 + 16 + 0} = \sqrt{21} \approx 4.583$$

$$\text{sim}(x, y) = \frac{19}{5.099 \times 4.583} = \frac{19}{23.368} = \mathbf{0.813} \text{ (Cosine Distance} = 1 - 0.813 = \mathbf{0.187})$$

Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Differentiate between OLTP and OLAP systems across 6 distinct technical dimensions. (8 Marks)

- Answer:**
- Primary Purpose:** OLTP supports day-to-day transactional business operations (INSERT/UPDATE); OLAP supports decision support, executive reporting, and analytical queries.
 - User Base:** OLTP is used by clerks, front-end staff, DBAs; OLAP is used by managers, data scientists, business analysts.
 - Data Orientation:** OLTP is application-oriented and operational; OLAP is subject-oriented (Sales, Revenue, Customer).
 - Database Design:** OLTP is highly normalized (3NF/BCNF) to prevent write anomalies; OLAP is denormalized (Star/Snowflake multidimensional schemas).
 - Access Patterns:** OLTP executes short, atomic read/write transactions; OLAP executes complex read-only aggregate queries spanning millions of rows.
 - Time Horizon:** OLTP holds current real-time snapshot; OLAP stores multi-year historical time-variant snapshots.

Q2. Why is Jaccard Coefficient preferred over Simple Matching Coefficient for asymmetric binary data? (5 Marks)

Answer: In asymmetric binary datasets (e.g., market basket analysis where thousands of items exist), the absence of an item (0 – 0 match) carries virtually zero predictive significance (e.g., two customers not buying caviar does not indicate similarity). SMC includes negative matches (t) in both numerator and denominator, causing two unrelated customers to appear artificially 99.9% similar. The **Jaccard Coefficient** excludes t entirely, focusing exclusively on joint positive co-occurrences (g), yielding a true measure of affinity.

Module II: Data Preprocessing & Processing — 6 Topics Syllabus

- | | |
|--|--|
| 1. Data Cleaning (Missing Values, Noise Binning, Regression, Outliers) | 2. Data Integration (Entity Identification, Chi-Square, Pearson, Spearman, Covariance) |
| 3. Data Transformation (Min-Max, Z-Score, Decimal Scaling, Box-Cox, Log) | 4. Data Reduction (PCA Derivation, SVD, Wavelets, Numerosity, Sampling, Compression) |
| 5. Data Discretization (Binning, Histograms, Cluster, ChiMerge Step-by-Step) | 6. Concept Hierarchy Generation (Categorical & Numeric 3–4–5 Rule) |

Topic 1: Data Cleaning (Noise, Missing Values & Outliers)

Real-world industrial datasets are inherently "dirty" — incomplete (lacking attribute values or containing only aggregate data), noisy (containing errors, outliers, or transmission corruptions), and inconsistent (containing discrepancies in codes, names, or measurement units).

Data cleaning routines attempt to fill in missing values, smooth out noise while identifying outliers, and correct inconsistencies.

1. Advanced Strategies for Handling Missing Attribute Values:

- 1. Ignore the Tuple:** Typically done when the target class label is missing in supervised classification tasks. This method is inefficient when the percentage of missing values per attribute varies considerably across records.
- 2. Fill in the Missing Value Manually:** High accuracy but extraordinarily time-consuming, expensive, and infeasible for large multi-gigabyte databases.
- 3. Use a Global Constant:** Replace all missing attribute values with a fixed label such as "Unknown" or a sentinel value ($-\infty$). Simple, but may lead the mining algorithm to infer false correlations among tuples sharing the global constant.
- 4. Use Attribute Mean or Median:**
 - Use the **Arithmetic Mean** for symmetric (Gaussian) continuous distributions.
 - Use the **Median** for highly skewed distributions (e.g., household income or real estate prices).
- 5. Use the Attribute Mean/Median for All Samples of the Same Class:** If classifying credit risk, fill missing salary with the mean salary of customers sharing the same credit classification label.
- 6. Use the Most Probable Value (Statistical Imputation):** Determine the missing value using linear regression, Bayesian networks, inference-based tools, or decision tree induction. Considered the most statistically rigorous approach.
- 7. K-Nearest Neighbor (KNN) Imputation:** Identifies the k most similar complete records based on distance metrics (e.g., Euclidean distance) and imputes the weighted average of the neighbors' values:

$$\hat{x}_i = \frac{\sum_{j \in N_k(i)} \frac{1}{d(x_i, x_j)} x_j}{\sum_{j \in N_k(i)} \frac{1}{d(x_i, x_j)}}$$

- 8. Multiple Imputation by Chained Equations (MICE):** A series of iterative regression models where each missing variable is modeled conditionally on other variables in the dataset over multiple imputation cycles.

2. Smoothing Noisy Data via Binning Techniques:

Binning methods smooth sorted data values by consulting their neighborhood (local values around each point):

Step-by-Step Solved Problem 1: Binning Smoothing Methods

Sorted Data for Price Attribute: 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34 ($n = 12$).

Step 1: Partition into 3 Equal-Frequency (Equal-Depth) Bins (Depth = 4 items/bin):

- Bin 1: [4, 8, 9, 15]
- Bin 2: [21, 21, 24, 25]
- Bin 3: [26, 28, 29, 34]

Step 2: Smoothing by Bin Means:

- Mean(Bin 1) = $\frac{4+8+9+15}{4} = \frac{36}{4} = 9 \implies$ Bin 1: [9, 9, 9, 9]
- Mean(Bin 2) = $\frac{21+21+24+25}{4} = \frac{91}{4} = 22.75 \implies$ Bin 2: [23, 23, 23, 23]
- Mean(Bin 3) = $\frac{26+28+29+34}{4} = \frac{117}{4} = 29.25 \implies$ Bin 3: [29, 29, 29, 29]

Step 3: Smoothing by Bin Boundaries (Min and Max of Each Bin):

- Bin 1 (Min = 4, Max = 15): $8 \rightarrow 4, 9 \rightarrow 15 \implies [4, 4, 15, 15]$
- Bin 2 (Min = 21, Max = 25): $24 \rightarrow 25 \implies [21, 21, 25, 25]$
- Bin 3 (Min = 26, Max = 34): $28 \rightarrow 26, 29 \rightarrow 26 \implies [26, 26, 26, 34]$

3. Regression & Outlier Clustering for Smoothing:

- **Linear Regression:** Fits data to a straight line $y = wx + b$. Where slope $w = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$ and intercept $b = \bar{y} - w\bar{x}$. Values are replaced by their predicted regression coordinates.
- **Multiple Linear Regression:** Fits multidimensional attributes to a hyper-plane $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.
- **Cluster Analysis for Outlier Detection:** Similar values are organized into clusters. Values falling outside major cluster boundaries (e.g. distance $> 3\sigma$ from centroid) are flagged as outliers.
- **Local Outlier Factor (LOF):** Measures local density deviation of a given object with respect to its surrounding k -nearest neighbors. An object with density substantially lower than its neighbors is identified as a local outlier.

Topic 2: Data Integration & Redundancy Correlation Analysis

Data Integration combines data from multiple heterogeneous sources into a coherent store. Key challenges include:

- **Entity Identification Problem:** Matching real-world entities across databases (e.g., ensuring `customer\_id` in DB1 matches `cust\_number` in DB2 using schema metadata).
- **Redundancy & Correlation:** An attribute may be redundant if it can be derived from another attribute. Redundancy leads to dimensional bloat, increased storage overhead, and algorithm bias.

1. Chi-Square (χ^2) Correlation Test for Nominal Attributes:

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}}, \quad \text{where } E_{ij} = \frac{\text{row_total}_i \times \text{col_total}_j}{N}$$

Step-by-Step Solved Problem 2: Chi-Square (χ^2) Contingency Table Test

Contingency Table (Gender vs. Fiction Book Preference):

Gender	Prefers Fiction	Prefers Non-Fiction	Row Total
Male	250 (Observed O_{11})	200 (Observed O_{12})	450
Female	50 (Observed O_{21})	100 (Observed O_{22})	150
Col Total	300	300	$N = 600$

Step 1: Compute Expected Frequencies (E_{ij}):

- $E_{11} = \frac{450 \times 300}{600} = 225$, $E_{12} = \frac{450 \times 300}{600} = 225$
- $E_{21} = \frac{150 \times 300}{600} = 75$, $E_{22} = \frac{150 \times 300}{600} = 75$

Step 2: Compute χ^2 Statistic:

$$\chi^2 = \frac{(250 - 225)^2}{225} + \frac{(200 - 225)^2}{225} + \frac{(50 - 75)^2}{75} + \frac{(100 - 75)^2}{75} = \frac{625}{225} + \frac{625}{225} + \frac{625}{75} + \frac{625}{75} = 2.78 + 2.78 + 8.33 + 8.33 = 22.22$$

Step 3: Hypothesis Testing: Degrees of freedom $\nu = (r - 1)(c - 1) = (2 - 1)(2 - 1) = 1$. The critical value of χ^2 at significance $\alpha = 0.001$ is 10.83. Since $22.22 > 10.83$, the hypothesis of independence is strongly rejected $\Rightarrow$ **Gender and Book Preference are statistically correlated!**

2. Correlation Coefficient (Pearson's r) for Numeric Data:

$$r_{A,B} = \frac{\sum_{i=1}^n (a_i - \bar{A})(b_i - \bar{B})}{(n - 1)s_A s_B} = \frac{\sum_{i=1}^n (a_i b_i) - n\bar{A}\bar{B}}{(n - 1)s_A s_B}$$

- $r > 0$: A and B are **positively correlated** (values of A increase as B increases).
- $r = 0$: A and B are **independent / uncorrelated**.
- $r < 0$: A and B are **negatively correlated** (as A increases, B decreases).

3. Spearman's Rank Correlation Coefficient (ρ):

Used for ordinal attributes or monotonic non-linear numeric data:

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}$$

Where $d_i = \text{rank}(a_i) - \text{rank}(b_i)$ is the difference between ranks of tuple i .

Step-by-Step Solved Problem: Spearman's Rank Correlation

Given Student Exam Ranks in Data Mining (X) and Machine Learning (Y):

Student	Rank X	Rank Y	$d_i = X - Y$	d_i^2
S1	1	2	-1	1
S2	2	1	1	1
S3	3	4	-1	1
S4	4	3	1	1
S5	5	5	0	0
Total	-	-	-	$\sum d_i^2 = 4$

Calculation ($n = 5$):

$$\rho = 1 - \frac{6 \times 4}{5(25 - 1)} = 1 - \frac{24}{120} = 1 - 0.20 = \mathbf{0.80}$$

Strong positive rank correlation ($\rho = 0.80$) between Data Mining and Machine Learning performance!

4. Covariance Between Numeric Attributes:

$$\text{Cov}(A, B) = E[(A - \bar{A})(B - \bar{B})] = \frac{\sum_{i=1}^n (a_i - \bar{A})(b_i - \bar{B})}{n - 1}$$

Note: $r_{A,B} = \frac{\text{Cov}(A,B)}{s_A s_B}$. If $\text{Cov}(A, B) > 0$, both attributes vary in the same direction.

Topic 3: Data Transformation (Normalization Formulations)

Data transformation converts the data into forms appropriate for mining, preventing attributes with initially large ranges (e.g., `Salary` in thousands) from dominating attributes with smaller ranges (e.g., `Age` in tens).

Method	Mathematical Formulation	Key Advantages & Tradeoffs
1. Min-Max Normalization	$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$	Preserves exact linear relationships among data values; maps values to $[0, 1]$; encounters "out-of-bounds" error on future data exceeding range.
2. Z-Score Normalization (Zero-Mean)	$v' = \frac{v - \mu_A}{\sigma_A}$	Transforms data to have mean $\mu = 0$ and standard deviation $\sigma = 1$; extraordinarily robust when min and max are unknown or extreme outliers exist.
3. Decimal Scaling	$v' = \frac{v}{10^j} \quad \text{where } j \text{ is smallest integer such that } \max(v') < 1$	Fast bit-shift operation; normalizes by moving decimal point.
4. Box-Cox Power Transform	$y^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(x) & \text{if } \lambda = 0 \end{cases}$	Parametric power transformation used to stabilize variance and normalize non-Gaussian positive distributions.
5. Log Transformation	$v' = \log_{10}(v) \quad \text{or} \quad \ln(v + 1)$	Compresses large values and stretches small values; ideal for right-skewed revenue/income curves.

Step-by-Step Solved Problem 3: Normalization Calculations

Given: Attribute `Income` with $\min = \$12,000$, $\max = \$98,000$, $\mu = \$54,000$, $\sigma = \$16,000$. Transform value $v = \$73,600$.

1. **Min-Max Normalization to Range** $[0.0, 1.0]$:

$$v' = \frac{73,600 - 12,000}{98,000 - 12,000} (1.0 - 0.0) + 0.0 = \frac{61,600}{86,000} = \mathbf{0.716}$$

2. **Z-Score Normalization:**

$$v' = \frac{73,600 - 54,000}{16,000} = \frac{19,600}{16,000} = \mathbf{1.225}$$

3. **Normalization by Decimal Scaling** ($\max = 98,000 \implies j = 5$ since $98,000/10^5 = 0.98 < 1$):

$$v' = \frac{73,600}{10^5} = \mathbf{0.736}$$

Topic 4: Data Reduction (PCA Mathematical Derivation & Numerosity)

Complex data analysis and mining on massive datasets may take an impractical amount of time. **Data Reduction** techniques obtain a reduced representation of the dataset that is much smaller in volume yet closely maintains the integrity of the original data.

1. Complete Step-by-Step Mathematical Derivation of PCA:

Worked Problem 4: Complete PCA Step-by-Step Matrix Derivation

Given 4 2D Data Points: $x_1 = (2, 4)$, $x_2 = (4, 6)$, $x_3 = (6, 8)$, $x_4 = (8, 10)$ ($n = 4$, $d = 2$).

1. Step 1: Compute Attribute Means:

$$\mu_1 = \frac{2 + 4 + 6 + 8}{4} = 5.0, \quad \mu_2 = \frac{4 + 6 + 8 + 10}{4} = 7.0$$

2. Step 2: Construct Zero-Mean Centered Matrix (X):

$$X = \begin{bmatrix} 2-5 & 4-7 \\ 4-5 & 6-7 \\ 6-5 & 8-7 \\ 8-5 & 10-7 \end{bmatrix} = \begin{bmatrix} -3 & -3 \\ -1 & -1 \\ 1 & 1 \\ 3 & 3 \end{bmatrix}$$

3. Step 3: Calculate Covariance Matrix ($C = \frac{1}{n-1} X^T X$):

$$C = \frac{1}{3} \begin{bmatrix} -3 & -1 & 1 & 3 \\ -3 & -1 & 1 & 3 \end{bmatrix} \begin{bmatrix} -3 & -3 \\ -1 & -1 \\ 1 & 1 \\ 3 & 3 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} = \begin{bmatrix} 6.67 & 6.67 \\ 6.67 & 6.67 \end{bmatrix}$$

4. Step 4: Solve for Eigenvalues ($\det(C - \lambda I) = 0$):

$$\det \begin{bmatrix} 6.67 - \lambda & 6.67 \\ 6.67 & 6.67 - \lambda \end{bmatrix} = (6.67 - \lambda)^2 - (6.67)^2 = \lambda^2 - 13.33\lambda = 0$$
$$\lambda(\lambda - 13.33) = 0 \implies \lambda_1 = 13.33, \quad \lambda_2 = 0.0$$

5. Step 5: Compute Eigenvector for Principal Component $\lambda_1 = 13.33$:

$$\begin{bmatrix} 6.67 - 13.33 & 6.67 \\ 6.67 & 6.67 - 13.33 \end{bmatrix} \begin{bmatrix} e_{11} \\ e_{12} \end{bmatrix} = \begin{bmatrix} -6.67 & 6.67 \\ 6.67 & -6.67 \end{bmatrix} \begin{bmatrix} e_{11} \\ e_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies e_{11} = e_{12}$$

$$\text{Normalized unit eigenvector: } e_1 = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} \approx \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}.$$

6. Step 6: Proportion of Total Variance Explained:

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{13.33}{13.33 + 0.0} = 100\%$$

The dataset can be compressed from 2 dimensions down to 1 dimension with **zero information loss!**

2. Discrete Wavelet Transforms (DWT) & Haar Decomposition:

DWT transforms a numeric vector X of length $N = 2^m$ into wavelet coefficients via hierarchical subband decomposition.

Step-by-Step Solved Problem: Haar Wavelet Decomposition

Given 8-Element Signal: $S = [2, 2, 4, 4, 6, 6, 8, 8]$.

1. **Level 1 Averages (Approximation):** $\left[\frac{2+2}{2}, \frac{4+4}{2}, \frac{6+6}{2}, \frac{8+8}{2}\right] = [2, 4, 6, 8]$.

Level 1 Differences (Detail): $\left[\frac{2-2}{2}, \frac{4-4}{2}, \frac{6-6}{2}, \frac{8-8}{2}\right] = [0, 0, 0, 0]$.

2. **Level 2 Averages:** $\left[\frac{2+4}{2}, \frac{6+8}{2}\right] = [3, 7]$.

Level 2 Differences: $\left[\frac{2-4}{2}, \frac{6-8}{2}\right] = [-1, -1]$.

3. **Level 3 Average (Overall Mean):** $\frac{3+7}{2} = 5$.

Level 3 Difference: $\frac{3-7}{2} = -2$.

4. **Full Haar Wavelet Transformed Vector:** $[5, -2, -1, -1, 0, 0, 0, 0]$. Zero detail coefficients can be truncated with zero reconstruction loss!

Topics 5 – 6: Data Discretization & Concept Hierarchy Generation

Data Discretization reduces the number of values for a given continuous attribute by dividing the range of the attribute into distinct intervals.

The ChiMerge (χ^2 Merge) Algorithm:

ChiMerge is a supervised, bottom-up discretization method based on the χ^2 test.

1. Sort continuous attribute values in ascending order. Each distinct value forms an initial interval.
2. For each pair of adjacent intervals, compute the χ^2 statistic based on class distribution.
3. Find the pair of adjacent intervals with the smallest χ^2 value (lowest difference in class labels).
4. If the minimum $\chi^2 < \text{threshold}$, merge the two intervals.
5. Repeat steps 2–4 until all adjacent intervals have $\chi^2 \geq \text{threshold}$ or number of intervals reaches maximum limit.

Concept Hierarchy Generation by the 3–4–5 Rule:

The **3–4–5 Rule** segments numeric intervals naturally into human-friendly chunks:

- If an interval covers 3, 6, 7, or 9 distinct units at the most significant digit, partition the range into **3 equal-width intervals** (3 for 3, 6, 9; 3 for 7 as 2 – 3 – 2).
- If an interval covers 2, 4, or 8 distinct units, partition the range into **4 equal-width intervals**.
- If an interval covers 1, 5, or 10 distinct units, partition the range into **5 equal-width intervals**.

Q1. Explain the step-by-step mathematical procedure of Principal Component Analysis (PCA) for data reduction. (10 Marks)

Answer: 1. **Standardize Input Data:** Given n data vectors of dimension d , normalize each attribute to zero mean: $x_{ij} \leftarrow x_{ij} - \bar{x}_j$.

2. **Calculate Covariance Matrix:** Compute $d \times d$ covariance matrix $C = \frac{1}{n-1} X^T X$, where $C_{jk} = \text{cov}(X_j, X_k)$.

3. **Compute Eigenvalues & Eigenvectors:** Solve characteristic equation $\det(C - \lambda I) = 0$ to find d eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$ and orthonormal eigenvectors $e_1, e_2, \dots, e_d$.

4. **Select Top k Components:** Choose k eigenvectors corresponding to largest eigenvalues such that cumulative variance explained $\frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j} \geq 0.95$.

5. **Project Data:** Transform original $n \times d$ matrix X onto k -dimensional orthogonal subspace: $Y = X \cdot E_k$, reducing dimensionality from d to k while preserving maximum variance.

Q2. Differentiate between Stratified Sampling and Simple Random Sampling with respect to data reduction. (6 Marks)

Answer: In Simple Random Sampling (SRS), every tuple in dataset D has an equal probability of selection ($1/N$). If data contains rare skewed classes (e.g., credit card fraud occurring in 0.1% of cases), SRS may completely omit all positive fraud instances in the sample.

In **Stratified Sampling**, the dataset is partitioned into mutually exclusive non-overlapping strata based on a key attribute (e.g., class label). Random samples are drawn independently from each stratum proportional to its representation, guaranteeing that rare classes are preserved in the reduced dataset.

Q3. Why is data preprocessing considered the most time-consuming phase in the KDD pipeline? (6 Marks)

Answer: Real-world operational data is collected from heterogeneous systems without unified validation. Studies show data preprocessing consumes 60–80% of total project time because it must resolve missing values, eliminate noise, reconcile disparate primary/foreign keys across schemas, convert incompatible data formats, remove redundant correlated attributes, and scale continuous numerical ranges without destroying underlying statistical relationships.

Q4. Compare Min-Max Normalization and Z-Score Normalization. When should each be chosen? (6 Marks)

Answer: 1. **Min-Max Normalization** bounds data strictly into $[0, 1]$, preserving exact original relative distances. Choose Min-Max when algorithms require inputs in a bounded range (e.g., Neural Network sigmoid activations, Image pixel values).

2. **Z-Score Normalization** transforms data to $\mu = 0, \sigma = 1$ without upper/lower bounds. Choose Z-Score when the absolute minimum and maximum are unknown or when severe outliers exist, as Min-Max would compress all normal data into a microscopic interval.

Module III: Data Warehousing & OLAP Technology — 9 Topics Syllabus

1. Basic Concepts of Data Warehouse (4 Pillars, OLTP vs OLAP)
2. Data Warehouse Modeling (Fact, Dimensions, Measures)
3. Data Cube & Lattice of Cuboids Formulation
4. OLAP Operations (Roll-up, Drill-down, Slice, Dice, Pivot)
5. Schema Design & SQL DDL (Star, Snowflake, Galaxy)
6. Data Warehouse Implementation (ROLAP, MOLAP, HOLAP, Indexing)
7. Generalization by Attribute-Oriented Induction (AOI Trace)
8. Data Cube Computation (Multiway Array, BUC, Star-Cubing)
9. Materialization Strategies (Full, Partial, Greedy Algorithm)

Topic 1: Basic Concepts of Data Warehouse

According to William H. Inmon, the acknowledged father of the data warehouse, a **Data Warehouse** is defined as:

William H. Inmon's Definitive Definition

*"A data warehouse is a **subject-oriented**, **integrated**, **time-variant**, and **non-volatile** collection of data in support of management's decision-making process."*

The Four Foundational Pillars:

- **1. Subject-Oriented:** Organized around major business subjects (e.g., Customer, Product, Supplier, Sales) rather than day-to-day transactional applications (e.g., Order Entry, Invoicing). Focuses on modeling and analysis for decision makers.
- **2. Integrated:** Built by unifying multiple heterogeneous data sources (RDBMS, flat files, online logs). Discrepancies in naming conventions, measurement scales, and encoding formats are resolved and cleansed during ETL (Extract-Transform-Load).
- **3. Time-Variant:** Data is stored as historical snapshots covering 5 to 10 years. Every key structure in a data warehouse explicitly contains an element of time (e.g., `Day\_ID`, `Month\_ID`, `Fiscal\_Year`).
- **4. Non-Volatile:** A data warehouse is physically separate from operational systems. Operational updates, deletions, and rollbacks do not occur. Only two data operations exist: **initial data loading** and **read-only analytical access**.

10-Point Technical Comparison: OLTP vs. OLAP

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Primary Users	Clerks, DBAs, Operational Engineers	Knowledge Workers, Business Analysts, Executives
System Function	Day-to-day transaction processing	Decision support, strategic trend analytics
DB Design	Highly normalized (3NF / BCNF)	Denormalized Multidimensional (Star / Snowflake)
Data Orientation	Application-oriented (Orders, Invoices)	Subject-oriented (Sales, Revenue, Customer)
Data Content	Current, real-time snapshot; detailed	Historical snapshots (5–10 yrs); summarized & detailed
Access Frequency	High frequency, short atomic read/write	Moderate frequency, complex long read-only queries
Unit of Work	Short simple transactions (INSERT/UPDATE)	Complex aggregation queries over millions of rows
Records Accessed	Tens to hundreds of records per query	Millions of records aggregated per query
Database Size	Gigabytes to tens of Gigabytes	Terabytes to Petabytes
Optimization Focus	High transaction throughput & zero lock contention	High query throughput & ultra-fast response time

Topic 2 & 5: Multidimensional Modeling & Warehouse Schemas

The data warehouse is modeled around a **Multidimensional Data Model** composed of:

- **Fact Table:** Contains numerical **Measures** (e.g., `dollars\_sold`, `units\_shipped`) and foreign keys referencing dimension tables.
 - *Additive Measures:* Can be summed along all dimensions (e.g., `dollars\_sold`).
 - *Semi-Additive Measures:* Can be summed along some dimensions but not others (e.g., `account\_balance` can be summed across branches, but not across time).
 - *Non-Additive Measures:* Cannot be summed along any dimension (e.g., `unit\_price`, ratios).
- **Dimension Tables:** Describe the context and perspectives of the facts (e.g., `Item`, `Time`, `Branch`, `Location`).

Schema Model	Design Architecture & Normalization Level	Key Performance & Query Tradeoffs
1. Star Schema	Central fact table surrounded by non-normalized (denormalized) dimension tables forming a star pattern.	Fast join queries; simple SQL navigation; slight data redundancy in dimension tables.
2. Snowflake Schema	Dimension tables are normalized into hierarchical sub-tables (e.g., `Item` splits into `Item`, `Subcategory`, `Category`).	Eliminates dimensional redundancy; reduces storage space; complex multi-table SQL joins reduce analytical query performance.
3. Fact Constellation (Galaxy Schema)	Multiple fact tables sharing common conformed dimension tables (e.g., `Sales_Fact` and `Shipping_Fact` both sharing `Time` and `Item`).	Sophisticated enterprise-wide model; supports cross-functional enterprise reporting.

Sample SQL DDL for Star Schema:

```
-- Fact Table DDL for AllElectronics Sales
CREATE TABLE Sales_Fact (
  time_key INT,
  item_key INT,
  branch_key INT,
  location_key INT,
  units_sold INT,
  dollars_sold DECIMAL(12, 2),
  avg_sales DECIMAL(10, 2),
  PRIMARY KEY (time_key, item_key, branch_key, location_key),
  FOREIGN KEY (time_key) REFERENCES Time_Dim(time_key),
  FOREIGN KEY (item_key) REFERENCES Item_Dim(item_key),
  FOREIGN KEY (branch_key) REFERENCES Branch_Dim(branch_key),
  FOREIGN KEY (location_key) REFERENCES Location_Dim(location_key)
);
```

Topic 3: The Data Cube & Lattice of Cuboids

A multidimensional data model represents data in the form of a **Data Cube**. In an n -dimensional space, data can be aggregated across all possible 2^n dimensional combinations, forming a hierarchical **Lattice of Cuboids**.

- **Base Cuboid (0-D Aggregation):** The most detailed, granular level containing all n dimensions.
- **Apex Cuboid (n -D Aggregation / All-Cuboid):** The highest level of generalization containing the grand total measure across all dimensions.
- **Total Number of Cuboids:** For a warehouse with n dimensions, where dimension i has a concept hierarchy with L_i levels:

$$T = \prod_{i=1}^n (L_i + 1)$$

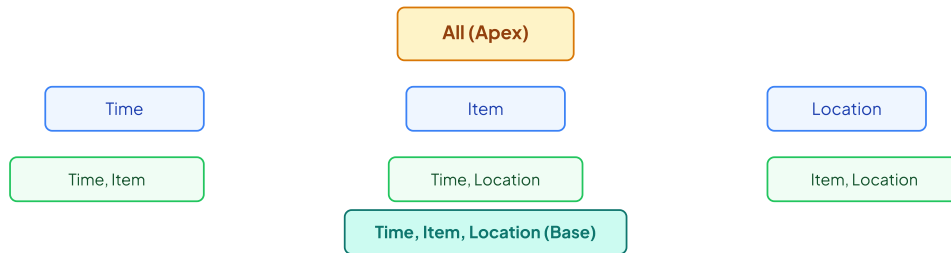


FIGURE 3.1: LATTICE OF CUBOIDS FOR A 3-DIMENSIONAL DATA CUBE ($2^3 = 8$ CUBOIDS)

Topic 4: Core OLAP Operations (The 5 Movements)

OLAP Operation	Formal Definition & Dimensional Manipulation	Concrete Business Example
1. Roll-Up (Drill-Up)	Performs aggregation along a dimension by climbing up a concept hierarchy or by dimension reduction.	Aggregating daily sales to quarterly sales; dropping the `Location` dimension to view total national revenue.
2. Drill-Down (Roll-Down)	Reverse of roll-up; navigates from less detailed data to more detailed data by stepping down a concept hierarchy or introducing a new dimension.	Expanding quarterly revenue by state down to monthly sales by individual retail store.
3. Slice	Performs a selection on a single dimension of the data cube, yielding a 2D sub-table.	Selecting `Time = "Q1 2026"`` to inspect all item sales across locations for that quarter.
4. Dice	Defines a sub-cube by performing a selection on two or more dimensions simultaneously.	Selecting `Time = "Q1 2026"`` AND `Location = "India"`` AND `Item = "Laptop"``.
5. Pivot (Rotate)	Rotates the data axes in multidimensional space to provide an alternative visual presentation.	Swapping rows (Locations) and columns (Item Categories) in a reporting spreadsheet.

Topic 6: Data Warehouse Server Architectures (ROLAP vs. MOLAP vs. HOLAP)

Server Architecture	Internal Storage & Implementation	Key Tradeoffs
1. ROLAP (Relational OLAP)	Stores multidimensional data directly inside relational tables using Star/Snowflake schemas; uses specialized SQL middleware to translate OLAP queries into SQL joins.	Highly scalable to petabyte volumes; leverages existing RDBMS; query performance can be slower on complex aggregations.
2. MOLAP (Multidimensional OLAP)	Stores data directly in multidimensional array storage structures; pre-calculates and materializes cuboids.	Blazing fast query response times; optimal for dense cubes; suffers storage explosion on sparse data cubes.
3. HOLAP (Hybrid OLAP)	Stores detailed granular base data in relational tables (ROLAP) and stores high-level aggregated summary cuboids in multidimensional arrays (MOLAP).	Combines the massive storage capacity of ROLAP with the high query performance of MOLAP.

Topic 7: Generalization by Attribute–Oriented Induction (AOI)

Attribute–Oriented Induction (AOI) is a target–class concept generalization technique that extracts summary rules from relational tables using concept hierarchies without expert manual intervention.

The AOI Algorithm: Two Core Generalization Operators

1. **Attribute Removal:** If an attribute has a large number of distinct values and no generalization concept hierarchy is available (e.g., `Social\_Security\_Number`, `Employee\_ID`), the attribute is removed from the generalized relation.
2. **Attribute Generalization:** If an attribute has an associated concept hierarchy and the number of distinct values exceeds a user-specified threshold, replace lower-level values with higher-level concept concepts until the attribute threshold is satisfied.
3. **Tuple Aggregation:** Merge identical generalized tuples and accumulate count/sum measures.

Step-by-Step Solved Problem: AOI on University Graduate Database

Initial Relation: $T = (\text{Name}, \text{Gender}, \text{Major}, \text{Birth_Place}, \text{GPA})$.

1. `Name` has 5,000 distinct values and no concept hierarchy $\implies$ **Attribute Removal**.
2. `Gender` has 2 distinct values $\leq \text{Threshold}(3)$ $\implies$ **Retain**.
3. `Major` has hierarchy (CSE, ECE, ME $\rightarrow$ Engineering; Physics, Chem $\rightarrow$ Science) $\implies$ **Generalize to Department**.
4. `Birth\_Place` has hierarchy (City $\rightarrow$ Province/State $\rightarrow$ Country) $\implies$ **Generalize to Country**.
5. `GPA` continuous numeric $\implies$ **Discretize to {Excellent, Good, Fair}**.
6. **Final Generalized Relation:** (Gender, Department, Country, GPA_Class, Count%).

Topic 8 – 9: Data Cube Computation & Materialization Strategies

1. Multiway Array Aggregation for Full Cube Computation:

Multiway Array Aggregation computes full data cubes by partitioning the multidimensional array into chunks that fit in main memory. By ordering the dimension aggregations such that smallest dimensions are kept in memory longest ($A \rightarrow B \rightarrow C$), the full cube is computed in a single pass over the base data.

2. Bottom-Up Computation (BUC):

BUC explores the lattice of cuboids bottom-up (starting from 1-D cuboids to n -D base cuboids). It exploits the **Iceberg Cube** pruning condition: if a cuboid cell has $\text{count} < \text{min_sup}$, BUC immediately prunes all descendant specialized cuboid cells, avoiding expensive aggregate queries.

3. Star-Cubing Algorithm:

Star-Cubing integrates the advantages of Star-Trees and Bottom-Up computation. It builds a shared prefix tree for the base cuboid and simultaneously compresses and computes shared sub-cuboid aggregates.

4. Materialization Tradeoffs:

- **Full Materialization:** Pre-computes all 2^n cuboids. Instantaneous query execution, but requires exponential disk space.
- **No Materialization:** Zero extra storage, but computing aggregates on-the-fly from terabyte base tables yields unacceptable query latencies.
- **Partial Materialization:** Selects a beneficial subset of cuboids to pre-compute using greedy benefit-cost optimization algorithms.

Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Draw and explain the 3-Tier Architecture of a Data Warehouse. (10 Marks)

Answer: 1. **Bottom Tier (Warehouse Database Server):** The relational database system holding the operational data extracted, transformed, and loaded via ETL tools (e.g., Informatica, DataStage). Stores metadata repository and data marts.

2. **Middle Tier (OLAP Server):** Implemented using ROLAP (relational engine with extended SQL joins) or MOLAP (multidimensional array storage engine) to execute multidimensional slice, dice, roll-up, and drill-down calculations.

3. **Top Tier (Front-End Client Tools):** Query and reporting tools, analysis tools, data mining algorithms, and executive dashboards.

Q2. Explain Bitmap Indexing and Join Indexing for OLAP query optimization. (8 Marks)

Bitmap Indexing: Used for attributes with low cardinality (few distinct values, e.g., 'Gender' or 'Marital_Status'). A bit vector is maintained for each distinct value where bit $k = 1$ if tuple k contains the value, 0 otherwise. Logical AND/OR/NOT bitwise operations execute in single CPU clock cycles.

Join Indexing: Pre-computes relationships between foreign keys in fact tables and primary keys in dimension tables, eliminating runtime Cartesian join overhead during analytical SQL execution.

Q3. What is an Iceberg Cube and how does BUC (Bottom-Up Computation) prune search space? (6 Marks)

Answer: An **Iceberg Cube** computes only those cuboid cells whose aggregate measure satisfies a user-specified minimum threshold (e.g., $\text{count} \geq \text{min_sup}$). In BUC, cuboids are processed bottom-up from apex downwards. If an aggregate cell count falls below **min_sup**, the Apriori downward closure property guarantees that none of its specialized descendant sub-cells can meet the threshold, allowing BUC to immediately prune the entire subtree.

Q4. Compare Enterprise Warehouse, Data Mart, and Virtual Warehouse. (6 Marks)

Enterprise Warehouse: Collects all information about subjects spanning the entire organization. Provides corporate-wide data integration across all business units.

Data Mart: Contains a subset of corporate-wide data that is of value to a specific department or group of users (e.g., Marketing Data Mart).

Virtual Warehouse: A set of views over operational databases. Does not require separate physical storage; easy to build but imposes high query load on operational databases.

Module IV: Frequent Pattern Mining & Association Analysis — 7 Topics Syllabus

1. Basic Concepts of Frequent Pattern Mining
2. Association Rules (Support & Confidence)
3. Correlation Analysis (Lift, Chi-Square, Kulczynski)
4. Frequent Itemset Mining Methods Landscape (Apriori, FP-Growth, ECLAT)
5. The Apriori Algorithm (Formal Proof, Join & Prune Steps)
6. Pattern-Growth Approach (FP-Growth, FP-Tree, Conditional Bases)
7. Interesting Pattern Evaluation Methods (Null-Invariance)

Topics 1 – 2: Market Basket Analysis & Association Rules

Frequent Patterns are itemsets, subsequences, or substructures that appear frequently in a transactional dataset (with frequency not less than a user-specified threshold).

Market Basket Analysis investigates customer buying habits by finding associations between the different items that customers place in their shopping baskets:

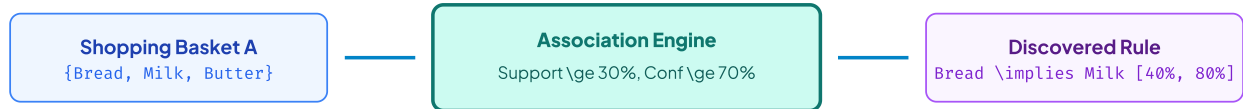


FIGURE 4.1: MARKET BASKET ASSOCIATION RULE EXTRACTION

Mathematical Formulations:

- **Itemset:** A set of items $X = \{i_1, i_2, \dots, i_k\}$. A k -itemset contains exactly k items.
- **Support Count ($\sigma(X)$):** The absolute number of transactions in database D containing itemset X :

$$\sigma(X) = |\{T \in D \mid X \subseteq T\}|$$

- **Relative Support (s):** Fraction of transactions containing X :

$$\text{Support}(X) = \frac{\sigma(X)}{|D|}$$

- **Association Rule:** An expression $A \implies B$ where $A \subset I, B \subset I$, and $A \cap B = \emptyset$.

$$\text{Support}(A \implies B) = P(A \cup B) = \frac{\sigma(A \cup B)}{|D|}$$

$$\text{Confidence}(A \implies B) = P(B \mid A) = \frac{\sigma(A \cup B)}{\sigma(A)} = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Topic 5: The Apriori Algorithm (Level-Wise Search)

The Apriori Downward Closure Property (Apriori Property)

Theorem: All non-empty subsets of a frequent itemset must also be frequent.

Formal Proof: Let I be an itemset and $I' \subseteq I$. Every transaction $T \in D$ that contains I must inherently contain all of its subsets I' (since $I \subseteq T \implies I' \subseteq T$). Therefore:

$$\sigma(I') = |\{T \in D \mid I' \subseteq T\}| \geq |\{T \in D \mid I \subseteq T\}| = \sigma(I)$$

If I is frequent ($\sigma(I) \geq \text{min_sup}$), then $\sigma(I') \geq \text{min_sup}$, proving that I' is also frequent.

Pruning Rule (Contrapositive): If any $(k-1)$ -subset of a candidate k -itemset C_k is infrequent ($< \text{min_sup}$), then C_k cannot be frequent and is immediately pruned before database scanning!

Two-Step Candidate Generation Mechanism:

1. **Join Step** ($L_{k-1} \bowtie L_{k-1} \rightarrow C_k$): Join two frequent $(k-1)$ -itemsets $l_1, l_2 \in L_{k-1}$ if their first $k-2$ items are identical and the last item of l_1 is lexicographically smaller than l_2 :

$$(l_1[1] = l_2[1]) \wedge \dots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$$

2. **Prune Step:** For each candidate $c \in C_k$, check all $(k-1)$ -subsets of c . If any subset $\notin L_{k-1}$, delete c from C_k .

Step-by-Step Complete Apriori Trace on 9 Transactions

Transaction Database (D):

TID	Items Purchased
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Parameter: $\min_sup = 2$ ($\approx 22.2\%$), $\min_conf = 70\%$.

Complete Iterative Trace:

1. Pass 1:

- $C_1 = \{\{I1\} : 6, \{I2\} : 7, \{I3\} : 6, \{I4\} : 2, \{I5\} : 2\}$.
- All items have count $\geq 2 \implies L_1 = C_1$.

2. Pass 2:

- Join $L_1 \bowtie L_1 \rightarrow C_2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I4\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}, \{I3, I4\}, \{I3, I5\}, \{I4, I5\}\}$.
- Scan DB $\rightarrow L_2 = \{\{I1, I2\} : 4, \{I1, I3\} : 4, \{I1, I5\} : 2, \{I2, I3\} : 4, \{I2, I4\} : 2, \{I2, I5\} : 2\}$.
- Infrequent items dropped: $\{I3, I4\} : 0, \{I3, I5\} : 1, \{I4, I5\} : 0$.

3. Pass 3:

- Join $L_2 \bowtie L_2 \rightarrow$ Candidates:
 - $\{I1, I2\} \bowtie \{I1, I3\} \rightarrow \{I1, I2, I3\}$. Subsets: $\{I1, I2\}, \{I1, I3\}, \{I2, I3\} \in L_2 \implies$ **Keep**.
 - $\{I1, I2\} \bowtie \{I1, I5\} \rightarrow \{I1, I2, I5\}$. Subsets: $\{I1, I2\}, \{I1, I5\}, \{I2, I5\} \in L_2 \implies$ **Keep**.
 - $\{I2, I3\} \bowtie \{I2, I4\} \rightarrow \{I2, I3, I4\}$. Subset $\{I3, I4\} \notin L_2 \implies$ **Pruned!**
- $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$.
- Scan DB $\rightarrow L_3 = \{\{I1, I2, I3\} : 2, \{I1, I2, I5\} : 2\}$.

4. Pass 4: Join $L_3 \bowtie L_3 \rightarrow \{I1, I2, I3, I5\}$. Subset $\{I2, I3, I5\} \notin L_3 \implies$ **Pruned!** Algorithm terminates.

Rule Generation from Frequent Itemset $\{I1, I2, I5\}$ ($\sigma = 2$):

- Rule $\{I1, I5\} \implies \{I2\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I1, I5)} = \frac{2}{2} = 100\% \geq 70\% \implies$ **StrongRule!**
- Rule $\{I2, I5\} \implies \{I1\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I2, I5)} = \frac{2}{2} = 100\% \geq 70\% \implies$ **StrongRule!**
- Rule $\{I5\} \implies \{I1, I2\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I5)} = \frac{2}{2} = 100\% \geq 70\% \implies$ **StrongRule!**
- Rule $\{I1, I2\} \implies \{I5\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I1, I2)} = \frac{2}{4} = 50\% < 70\% \implies$ **Rejected!**

Topic 4: Vertical Data Format (ECLAT Algorithm)

ECLAT (Equivalence Class Transformation) mines frequent itemsets using a **Vertical Data Format** where each item is associated with its **TID-list** (list of transaction IDs containing the item):

Item	TID-List	Support Count
I1	{T100, T400, T500, T700, T800, T900}	6
I2	{T100, T200, T300, T400, T600, T800, T900}	7
I3	{T300, T500, T600, T700, T800, T900}	6
I4	{T200, T400}	2
I5	{T100, T800}	2

TID-List Intersections for 2-Itemsets:

- $\text{TID}(I1, I2) = \text{TID}(I1) \cap \text{TID}(I2) = \{T100, T400, T800, T900\} \implies \text{Count} = 4 \geq 2$
- $\text{TID}(I1, I3) = \{T500, T700, T800, T900\} \implies \text{Count} = 4 \geq 2$
- $\text{TID}(I1, I5) = \{T100, T800\} \implies \text{Count} = 2 \geq 2$
- $\text{TID}(I2, I3) = \{T300, T600, T800, T900\} \implies \text{Count} = 4 \geq 2$
- $\text{TID}(I2, I4) = \{T200, T400\} \implies \text{Count} = 2 \geq 2$
- $\text{TID}(I2, I5) = \{T100, T800\} \implies \text{Count} = 2 \geq 2$

Topic 6: The FP-Growth Algorithm (Mining Without Candidate Generation)

FP-Growth (Frequent Pattern Growth) overcomes the major performance bottlenecks of the Apriori algorithm (repeated full database scans and huge candidate itemset generation C_k).

The Two-Step FP-Growth Architecture:

1. Step 1: Construct the Compact FP-Tree:

- Scan database once to compute support counts of 1-itemsets; sort frequent items in descending order of support ($I2 : 7 > I1 : 6 > I3 : 6 > I4 : 2 > I5 : 2$).
- Scan database a second time. For each transaction, insert items into a shared prefix tree (FP-Tree). Shared transaction prefixes share branch nodes, incrementing branch counts.
- Construct the **Header Table** containing item heads and horizontal linked-list node pointers traversing all identical items in the tree.

2. Step 2: Mine the FP-Tree Recursively (Bottom-Up):

We extract Conditional Pattern Bases and construct Conditional FP-Trees starting from the bottom of the Header Table:

Item	Conditional Pattern Base	Conditional FP-Tree	Frequent Patterns Generated
I5	$\{I2, I1 : 1\}, \{I2, I1, I3 : 1\}$	$\langle I2 : 2, I1 : 2 \rangle$	$\{I2, I5 : 2\}, \{I1, I5 : 2\}, \{I2, I1, I5 : 2\}$
I4	$\{I2 : 1\}, \{I2, I1 : 1\}$	$\langle I2 : 2 \rangle$	$\{I2, I4 : 2\}$
I3	$\{I2, I1 : 2\}, \{I2 : 2\}, \{I1 : 2\}$	$\langle I2 : 4, I1 : 4 \rangle$	$\{I2, I3 : 4\}, \{I1, I3 : 4\}, \{I2, I1, I3 : 2\}$
I1	$\{I2 : 4\}$	$\langle I2 : 4 \rangle$	$\{I2, I1 : 4\}$

Topic 3 & 7: Interesting Pattern Evaluation Metrics

Lift Correlation Formulation:

$$\text{Lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)} = \frac{\text{Confidence}(A \implies B)}{\text{Support}(B)}$$

Metric Name	Mathematical Formula	Interpretation & Properties
Lift	$\text{Lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)}$	> 1: Positively correlated. = 1: Independent. < 1: Negatively correlated.
Max-Confidence	$\text{Max_Conf}(A, B) = \max(P(A \mid B), P(B \mid A))$	Null-invariant metric.
Kulczynski (Kulc)	$\text{Kulc}(A, B) = \frac{1}{2} (P(A \mid B) + P(B \mid A))$	Arithmetic mean of conditional probabilities; Null-invariant (unaffected by total transaction size $ D $).
Imbalance Ratio (IR)	$\text{IR}(A, B) = \frac{ \text{supp}(A) - \text{supp}(B) }{\text{supp}(A) + \text{supp}(B) - \text{supp}(A \cup B)}$	Measures asymmetry in itemset support; used in tandem with Kulc.
Cosine (Itemsets)	$\text{Cosine}(A, B) = \frac{P(A \cup B)}{\sqrt{P(A)P(B)}} = \sqrt{P(A \mid B)P(B \mid A)}$	Geometric mean of conditional probabilities; Null-invariant.

Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Compare Apriori and FP-Growth algorithms across 5 critical engineering metrics. (8 Marks)

1. **Candidate Generation:** Apriori generates millions of candidate k -itemsets (C_k); FP-Growth generates zero candidates.
2. **Database Scans:** Apriori requires $k_{\max} + 1$ full scans of D ; FP-Growth requires exactly 2 database scans.
3. **Data Structure:** Apriori uses flat transactional arrays and hash trees; FP-Growth builds a highly compressed prefix tree with linked header tables.
4. **Memory Usage:** Apriori requires huge RAM for candidate sets C_k ; FP-Growth is memory compact via shared transaction prefixes.
5. **Mining Strategy:** Apriori is breadth-first (level-wise); FP-Growth is depth-first (divide-and-conquer).

Q2. Why is Support-Confidence framework insufficient for evaluating pattern interestingness? (6 Marks)

Answer: A rule can have high support and high confidence yet be completely uninteresting or deceptive. For example, in a grocery store where 75% of all customers purchase coffee, the rule $\text{Tea} \implies \text{Coffee}$ might have 75% confidence simply because coffee is popular overall, even though drinking tea might actually decrease the likelihood of buying coffee (negative correlation). Metrics like **Lift** and **Kulczynski** normalize for base individual item probabilities, exposing true causal affinity.

Q3. What is Null-Invariance in pattern evaluation and why is it crucial for massive transaction databases? (6 Marks)

Answer: A pattern interestingness measure M is **Null-Invariant** if its value is completely unaffected by the number of transactions that contain neither item A nor item B ($0 - 0$ negative transactions). In web transaction logs and massive e-commerce databases with millions of total transactions, the number of null transactions is astronomical. Non-null-invariant measures (like χ^2 and Lift) become severely distorted by total transaction count $|D|$, whereas null-invariant measures (like Kulczynski and Cosine) accurately measure true item affinity regardless of database size.

Module V: Advanced Pattern Mining & Applications — 10 Topics Syllabus

- | | |
|---|---|
| 1. Pattern Mining Road Map & Frontier Challenges | 2. Multilevel Pattern Mining (Uniform vs Reduced Support, Redundancy) |
| 3. Multidimensional Pattern Mining (Interdimension & Hybrid Rules) | 4. Constraint-Based Frequent Pattern Mining (5 Classes & Mathematical Proofs) |
| 5. Mining High-Dimensional Data & Top-k Patterns | 6. Mining Colossal Patterns (Pattern-Fusion Algorithm & Core Patterns) |
| 7. Mining Compressed Patterns (Closed vs Maximal Frequent Itemsets) | 8. Mining Approximate & Noisy Patterns |
| 9. Sequential & Graph Pattern Mining (GSP, PrefixSpan, gSpan) | 10. Real-World Pattern Applications across 5 Key Industries |

Topic 1: Advanced Pattern Mining Road Map

Advanced pattern mining extends standard single-dimensional market basket analysis into complex, multidimensional, constrained, and high-dimensional spaces:

1. **Complex Data Structures:** Moving from sets of items to sequential patterns (time-ordered sequences), structured subtrees, spatial topologies, and graph substructures.
2. **Multiple Abstraction Levels:** Mining across hierarchical taxonomy levels without generating millions of redundant rules.
3. **Multidimensional & Quantitative Predicates:** Handling relational tables with categorical and continuous numeric attributes simultaneously.
4. **Constraint-Based Pushing:** Pushing application-specific constraints into candidate generation and prefix tree construction to avoid combinatorial explosion.
5. **High-Dimensionality & Colossal Patterns:** Overcoming the 2^{100} search space barrier when patterns contain tens or hundreds of items.

Topic 2: Multilevel Pattern Mining

Items in real-world retail catalogs naturally form concept hierarchies (e.g., **Computer** → **Laptop** → **Dell XPS 15**).

Support Strategy	Mechanism & Threshold Settings	Key Tradeoffs
1. Uniform Support	The same minimum support threshold (e.g., $\text{min_sup} = 5\%$) is applied at all abstraction levels.	Simple to implement; misses rare granular items at lower levels while producing trivial rules at top levels.
2. Reduced (Progressive) Support	Higher support thresholds at top levels ($\text{min_sup} = 5\%$ for 'Computer'), progressively lower thresholds at granular levels ($\text{min_sup} = 0.5\%$ for 'Dell XPS 15').	Discovers rare low-level patterns; avoids flooding top levels with uninteresting rules.
3. Group-Based Support	Custom support thresholds set per product category (e.g., lower threshold for diamond rings, higher for milk).	Reflects business domain reality; requires domain expert configuration.

Redundant Rule Filtering:

A rule r_1 is redundant if it is an ancestor of rule r_2 across the hierarchy and its confidence is close to the expected value derived from r_2 :

$$\text{Expected Confidence}(r_1) \approx \text{Confidence}(r_2)$$

Such ancestor rules convey no additional novel knowledge and are automatically filtered out.

Topic 3: Multidimensional & Quantitative Association Rules

Rule Paradigm	Definition & Predicate Structure	Example Rule
Single-Dimensional (Boolean)	Distinct predicates occur multiple times with different values (only one dimension `buys`).	$\text{buys}(X, \text{"Milk"}) \implies \text{buys}(X, \text{"Bread"})$
Interdimension Association Rule	Each predicate appears at most once in the rule across multiple distinct dimensions.	$\text{Age}(X, \text{"20..29"}) \wedge \text{Occupation}(X, \text{"Student"}) \implies \text{Buys}(X, \text{"Laptop"})$
Hybrid Association Rule	Combines multiple predicates where one or more predicates may appear repeatedly.	$\text{Age}(X, \text{"20..29"}) \wedge \text{Buys}(X, \text{"Laptop"}) \implies \text{Buys}(X, \text{"Backpack"})$

Quantitative Association Rules Mining Approaches:

1. **Static Discretization:** Partition continuous numeric attributes into predefined intervals (e.g., using equidepth binning) prior to mining.
2. **Dynamic Discretization (Grid-Based Clustering):** Discretizes quantitative dimensions dynamically during mining to maximize rule support and confidence (e.g., ARCS algorithm).

Topic 4: Constraint-Based Frequent Pattern Mining

Rather than mining unconstrained millions of patterns and filtering afterward, **Constraint-Based Mining** pushes user-specified constraints deeply into the mining algorithm (Apriori candidate generation and FP-Growth tree construction) to prune search branches early.

Constraint Class	Mathematical Property & Behavior	Concrete Example & Pushing Strategy
1. Antimonotonic	If an itemset S violates constraint C , all supersets $S' \supset S$ also violate C .	$\text{sum}(S.\text{price}) \leq 500, \min(S.\text{price}) \geq 20$. Push: Prune candidate branches immediately.
2. Monotonic	If an itemset S satisfies constraint C , all supersets $S' \supset S$ also satisfy C .	$\text{sum}(S.\text{price}) \geq 1000, \max(S.\text{price}) \geq 50$. Push: Once S satisfies C , skip testing all its supersets.
3. Succinct	All itemsets satisfying C can be explicitly enumerated using an analytical formula without candidate generation.	$\min(S.\text{price}) \leq 10, \text{Item.type} = \text{"Electronics"}$. Push: Filter items upfront before mining.
4. Convertible	A constraint that is neither monotonic nor antimonotonic, but converts into one when items are arranged in a specific sorting order.	$\text{avg}(S.\text{price}) \leq 50$ (Convertible antimonotonic if items are sorted by descending price).

Step-by-Step Formal Mathematical Proofs of Constraint Properties

Proof 1: Prove that $\text{sum}(S.\text{price}) \leq 500$ is Antimonotonic (assuming non-negative item prices $p_i \geq 0$).

Let S be an itemset violating the constraint $\implies \text{sum}(S.\text{price}) > 500$.

For any proper superset $S' \supset S$, $\text{sum}(S'.\text{price}) = \text{sum}(S.\text{price}) + \sum_{i \in S' \setminus S} p_i$.

Since $p_i \geq 0$, $\sum_{i \in S' \setminus S} p_i \geq 0 \implies \text{sum}(S'.\text{price}) \geq \text{sum}(S.\text{price}) > 500$.

Thus, every superset S' violates the constraint $\implies$ **Antimonotonic Property Proven!**

Proof 2: Prove that $\max(S.\text{price}) \geq 50$ is Monotonic.

Let S be an itemset satisfying the constraint $\implies \max(S.\text{price}) \geq 50$.

For any proper superset $S' \supset S$, $\max(S'.\text{price}) = \max(\max(S.\text{price}), \max((S' \setminus S).\text{price})) \geq \max(S.\text{price}) \geq 50$.

Thus, every superset S' automatically satisfies the constraint $\implies$ **Monotonic Property Proven!**

Topics 5 – 7: Mining High-Dimensional, Colossal & Compressed Patterns

1. Closed vs. Maximal Frequent Itemsets:

- **Closed Frequent Itemset:** An itemset X is closed if there exists no proper superset $Y \supset X$ such that $\text{Support}(Y) = \text{Support}(X)$. It provides a **lossless representation** of all frequent itemsets and their exact support counts.
- **Maximal Frequent Itemset (Max-Itemset):** An itemset X is maximal frequent if X is frequent and no proper superset $Y \supset X$ is frequent. It provides a compact **lossy representation** (the exact support of proper subsets is lost, only known to be $\geq \text{min_sup}$).

Step-by-Step Solved Problem: Closed vs. Maximal Frequent Itemsets

Given Frequent Itemsets with Support Counts:

- $L_1 : \{A\} : 4, \{B\} : 4, \{C\} : 3, \{D\} : 2$
- $L_2 : \{A, B\} : 4, \{A, C\} : 2, \{B, C\} : 2$
- $L_3 : \{A, B, C\} : 2$

Analysis:

1. Itemset $\{A\}$ has support 4. Superset $\{A, B\}$ has support 4 $\implies \{A\}$ is **NOT closed**.
2. Itemset $\{B\}$ has support 4. Superset $\{A, B\}$ has support 4 $\implies \{B\}$ is **NOT closed**.
3. Itemset $\{A, B\}$ has support 4. Proper superset $\{A, B, C\}$ has support 2 (< 4) $\implies \{A, B\}$ is **CLOSED**.
4. Itemset $\{A, B, C\}$ has support 2. No frequent superset exists $\implies \{A, B, C\}$ is **CLOSED and MAXIMAL**.
5. Itemset $\{D\}$ has support 2. No frequent superset exists $\implies \{D\}$ is **CLOSED and MAXIMAL**.

2. Mining Colossal Patterns via Pattern-Fusion:

When datasets contain colossal patterns (e.g., frequent itemsets of length 50 to 100 items), traditional level-wise and pattern-growth methods collapse because exploring the 2^{100} search space is computationally intractable. **Pattern-Fusion** jumps across the search space by fusing smaller core frequent patterns into large colossal patterns in single leaps rather than incremental 1-item extensions.

Topic 9: Sequential & Graph Pattern Mining (PrefixSpan & gSpan)

1. Sequential Pattern Mining (PrefixSpan):

A **Sequence Database** stores ordered lists of itemsets associated with timestamps (e.g., customer purchase sequences: $\langle (a)(abc)(df)(d) \rangle$).

- **PrefixSpan (Prefix-projected Sequential Pattern Mining):** Mines sequences without candidate generation by recursively projecting sequence databases into smaller projected databases based on frequent prefixes.
- Avoids candidate generation and exhibits linear scalability with respect to database size.

2. Graph Pattern Mining (gSpan):

Discovers frequent subgraphs in chemical compounds, biological networks, and social graphs. **gSpan** uses a canonical **DFS Code Dictionary** to uniquely identify and order graphs, pruning isomorphic duplicate candidates without expensive subgraph isomorphism testing.

Topic 10: Real-World Industrial Applications

1. **Retail & E-Commerce:** Market basket cross-selling, automated recommendation systems ("Customers who bought this also bought..."), promotional bundle optimization.
2. **Web Analytics & Clickstream Mining:** Discovering common web navigation paths to optimize website UI layouts and personalize dynamic landing pages.
3. **Healthcare & Bioinformatics:** Mining co-occurring genetic mutations, drug-drug interaction contraindications, diagnostic clinical pathways.
4. **Software Engineering & Bug Localization:** Mining execution traces of passing vs. failing test cases to identify fault-inducing code dependencies.
5. **Financial Fraud Detection:** Discovering synchronized multi-account transaction patterns characteristic of money laundering.

Top BIT Mesra Exam Questions & Answers (Module V)

Q1. Prove why the constraint $\text{sum}(S.\text{price}) \leq 500$ is Antimonotonic assuming non-negative prices. (6 Marks)

Proof: Let S be an itemset such that $\text{sum}(S.\text{price}) > 500$ (violating the constraint). For any superset $S' \supset S$, since item prices are non-negative ($p_i \geq 0$), $\text{sum}(S'.\text{price}) = \text{sum}(S.\text{price}) + \sum_{i \in S' \setminus S} p_i \geq \text{sum}(S.\text{price}) > 500$. Therefore, every superset S' is guaranteed to violate the constraint. By definition, $\text{sum}(S.\text{price}) \leq 500$ is **Antimonotonic**.

Q2. Explain the core mechanism of Pattern-Fusion for mining colossal patterns. (8 Marks)

Answer: Colossal patterns are massive patterns (length ≥ 50) that cannot be mined by traditional bottom-up $k \rightarrow k + 1$ extensions due to the combinatorial explosion of 2^{50} sub-patterns. Pattern-Fusion samples small, bounded *core patterns*, computes their equivalence classes, and leaps directly across the lattice by fusing multiple disjoint core patterns into colossal patterns in single steps, bypassing exponential intermediate levels.

Q3. Why is mining Closed Frequent Itemsets preferred over mining all frequent itemsets? (6 Marks)

Answer: If an itemset has 100 items, it contains $2^{100} - 1 \approx 10^{30}$ frequent sub-itemsets. Storing and mining all of them causes combinatorial explosion. The set of **Closed Frequent Itemsets** contains only those itemsets with no superset having the same support. It provides a 100% **lossless compression** of the frequent pattern space: the exact support of every single frequent itemset can be derived directly from the closed itemsets with zero information loss, while dramatically reducing memory consumption and mining runtime.

Q4. Differentiate between PrefixSpan and GSP algorithms for sequential pattern mining. (6 Marks)

GSP (Generalized Sequential Patterns): An Apriori-like level-wise algorithm that generates candidate sequences C_k from L_{k-1} and makes repeated full scans of the sequence database.

PrefixSpan (Prefix-projected Pattern Growth): A pattern-growth method that projects the sequence database into smaller suffix databases based on frequent prefixes, mining sequential patterns recursively without candidate generation.

Q5. Explain how quantitative association rules are mined using grid-based clustering. (6 Marks)

Answer: In grid-based clustering (e.g., ARCS system), continuous numeric dimensions (e.g., Age and Salary) are partitioned into 2D grid cells. Neighboring grid cells with high data density that satisfy minimum support are dynamically clustered into convex bounding polygons. Association rules are then extracted from these clustered regions, avoiding artificial rigid boundary partitions produced by standard static binning.